P0193R0 Where is Vectorization in C++?

Author: JF Bastien
Contact: jfb@google.com
Author: Hans Boehm
Contact: hboehm@google.com

Date: 2016-01-21
URL: https://github.com/jfbastien/papers/blob/master/source/P0193R0.rst

The C++ Standards Committee's Concurrency and Parallelism sub-group (WG21/SG1) has considered multiple approaches to user-aided vector programming and Single-Instruction Multiple-Data (SIMD) computations. The Committee's Game Development & Low Latency sub-group (WG21/SG14) has expressed interest in participating in this design as well. This paper documents the state of SG1's exploration with the goal of moving the Committee towards a Technical Specification in the near future.

This paper doesn't deal with auto-vectorization in any form.

Design Approach

SG1 has prefered library-only solutions for user-aided vector programming. These changes often integrate into the language more cleanly because they don't interact with the rest of the language and don't introduce onerous single-purpose syntax. This also allows the library implementation to use vendor-specific extensions in a manner that is seamless to the developer, giving leeway to implementations and allowing them to change their extensions over time without needing to standardize a particular implementation approach. When implementors peek under the hood the solution isn't library-only, but the Standard presents C++ users with what appears as a library-only solution.

A similar design approach was taken for the `<atomic>` library (as was the original motivation for a library-only design for coroutines). In the `<atomic>` case, interoperability with C was a strong design consideration, allowing `std::atomic` to be implemented in terms of `_Atomic` or vice versa, and allowing a program to use both correctly. A library approach allowed the Standard to reason in terms of objects and their lifetime instead of C's more error-prone per-access specification where each access has to be consistent lest it cause undefined behavior. It also allows implementations to specify storage for `std::atomic<T>` which differs from `T`'s storage, and which may have a different ABI. These library types further enable developers to use common generic programming patterns. We've found that some generic programming facilities were missing, such as constexpr atomic<T>::is_always_lock_free, and the library's design has made their addition simple.

The Committee in general has a strong leaning towards proposals which standardize existing practice. Implementation experience and usage experience are key. For vector programming, SG1 wants a Technical Specification which has at least one solid implementation, and would rather standardize something with multiple implementations and sustained usage. In the case of vector programming, platform portability and vendor buy-in matters much more than for most other C++ language or library features: the features are intended to work well on a variety of architectures with different fixed-width SIMD widths, with variable-width capabilities, and to degrade gracefully to scalar operation on architectures without vector capabilities.

The solution should work well for developers who target:

1. Multiple different ISAs with the same code base.
2. Multiple architectural implementations of the same ISA, sometimes supporting wider vectors in newer revisions.
3. A single architectural implementation (such as HPC or consoles) to obtain better performance by using their compiler's tuning flags such as `-march`.
4. Separate compilation of different source files, with the ability to provide vector code in libraries.

Vector programming requiring specific code-generation techniques such as Just-in-Time code generation would be a very unusual change for C++. Ahead-of-Time compilation is also frequently used, which doesn't preclude implementations from using other approaches if they desire.

Some in the committee believe that the amount of code generated by a compiler should not explode when new features are used, whereas others find the size trade-off acceptable. Requiring functions to be compiled multiple times is a no-go for some developers, especially if other language or library features interact in a combinatorial manner. One design point SG1 has considered is giving the developer control over the code size tradeoffs, as is possible with OpenMP.

Existing practice takes two approaches to user-aided vector programming:

1. High-level library focusing on APIs and their operations on data ranges.
2. Low-level vector types and operations on `N` scalars.

The former is typically easy to use while sometimes achieving optimal speedups. The later is often harder to use but grants developers more control, can provide significant speedups, and is usable to build some higher-level libraries. SG1 is comfortable moving forward with one Technical Specification in each category.

Open Design Areas

Vectorization is inherently re-association and therefore interacts with floating-point results. Beyond re-association, the solutions which SG1 has explored have not exposed architectural differences in the result's precision, yet some applications benefit from significant speedups when some precision is abandoned. It is usually up to the developer to figure out whether their application remains correct with

the loss of precision. It is unclear whether SG1 is comfortable exploring such vector programming opportunities at this point in time. For example:

- Image crossfade can be accelerated without loss of precision by using some architecture-specific instructions, or by dropping the least-significant bits when blending.
- Some architectures provide vector floating-point reciprocal and reciprocal square root approximation instructions, whereas in other architectures fast floating-point inverse square root can be obtained using integer multiply by a constant.
- Some architectures provide vectorized 16-bit floating-point, in some cases simply as a storage type whereas in other cases most arithmetic is supported on half-precision floating-point.
- Some architecture support Fused-Multiply-Add which performs one less rounding that the unfused equivalent, and is sometimes faster or less power-demanding than the unfused equivalent. Floating-point contraction is meant to address this, but the support for it isn't generally useful.

A core optimization which hasn't directly been addressed directly by current vector programming proposals is how to restructure data structures and algorithms to better fit the current architecture's SIMD width and memory hierarchy. C++ data structures have a fixed layout at a very stage in the compiler's pipeline, yet in many cases it is beneficial to change Array-of-Structures to Structures-of-Arrays. Low-level vector types as well as proposed reflection features (c.f. WG21/SG7) can be used to synthesize optimal data structures. This requires each programmer to express data structure layout transformations which are suitable to their algorithms and target architectures.

Latest Papers

The following papers are the latest ones being considered by SG1 with respect to vector programming. Other papers are obsolete, have been abandoned, or haven't been presented to SG1 recently.

Some of the following papers are explored in more details in the next sections.

- The Parallelism TS has a similar API as the STL's `<algorithm>` header. These algorithms are composable by the developer, who provides iterators to the data and, in some cases, function objects.
  - DTS Ballot Document.
  - Should be Standardized.
  - Template Library for Index-Based Loops.
  - Vector and wavefront policies.
  - Light-Weight Execution Agents discusses forward progress guarantees, with special attention to the Parralelism TS.
- The SIMD Types proposal exposes types which are fixed-width as well as `typedef` for wider vectors as appropriate for the target architecture.
  - The Vector Type & Operations.
  - The Mask Type & Write-Masking.
  - ABI Considerations.
  - Example: Matrix Multiplication.
  - A Proposal to add Single Instruction Multiple Data Computation to the Standard Library pre-dates the The Vector Type & Operations proposal and has similar API principles with different API choices and naming. The authors are discussing collaboration since The Vector Type & Operations proposal has received strong support from SG1.
- Dynamic memory allocation for over-aligned data allows allocating aligned data.

Parallelism TS Details

Of note in the Parallelism TS is the `par_vec` execution policy from section 2.6 [parallel.execpol.vec]:

> The class `class parallel_vector_execution_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be vectorized and parallelized.
>
> This execution policy is defined as follows in section 4.1.2 [parallel.alg.general.exec]:
>
> The invocations of element access functions in parallel algorithms invoked with an execution policy of type `parallel_vector_execution_policy` are permitted to execute in an unordered fashion in unspecified threads, and unsequenced with respect to one another within each thread. [Note: This means that multiple function object invocations may be interleaved on a single thread. — end note]
>
> [Note: This overrides the usual guarantee from the C++ standard, Section 1.9 [intro.execution] that function executions do not interleave with one another. — end note]
>
> Since `parallel_vector_execution_policy` allows the execution of element access functions to be interleaved on a single thread, synchronization, including the use of mutexes, risks deadlock. Thus the synchronization with `parallel_vector_execution_policy` is restricted as follows:
>
> A standard library function is vectorization-unsafe if it is specified to synchronize with another function invocation, or another function invocation is specified to synchronize with it, and if it is not a memory allocation or deallocation function. Vectorization-unsafe standard library functions may not be invoked by user code called from `parallel_vector_execution_policy` algorithms.
>
> [Note: Implementations must ensure that internal synchronization inside standard library routines does not induce deadlock. — end note]

Template Library for Index-Based Loops Details

The proposal adds the following to the Parallelism TS:

- `for_loop` and `for_loop_strided`.
- `reduction`, `reduction_plus`, `reduction_mutiplies`, …
- `induction`.

Vector and wavefront policies Details

The proposal adds two new execution policies to the Parallelism TS:

- `unseq_execution_policy`.
- `vec_execution_policy`.

This paper is contentious in SG1 because examples such as the following have `par_vec` data races:

```
for_loop(vec, 0, n, [&](int i) {
   y[i] += y[i+1];
});
```

In the implicit wavefront policy, this will work as expected: The load is sequenced before the store, and the loaded location is only overwritten by a later iteration. Operations that both appear earlier in the loop body (in the sequenced before sense) and in an earlier (or same) iteration than in another operation remain ordered in this model. This is the classic model followed by many existing vectorizing compilers.

In the explicit wavefront model, this requires a new kind of ordering barrier to explicitly ensure this ordering.

Three alternatives were discussed by a few SG1 members, outside of a full SG1 meeting. The first alternative is prefered by the small group at this point in time.

1. The implicit model continues to make some slightly uncomfortable, but there is agreement that we should nonetheless proceed with it. It is clearly the center of existing practice. And the negatives seem to be more along the lines of vague discomfort rather than precisely definable objections.

   The fundamental issue with this model is that it introduces a context in which certain otherwise safe compile transformations can no longer be applied by a compiler before the code is vectorized. Before the code is vectorized, the sequenced-before relation must, in the general case, be preserved. The compiler is no longer allowed to reorder ordinary assignments touching different memory locations. In other contexts such restrictions only arise in the presence of synchronization or volatile operations. For a compiler that immediately vectorizes before performing other transformations, this is not an issue. The current belief is that that this does commonly impact compiler structure.

   This transformation restriction of course also applies to the user: reordering independent operations in a vector context affects semantics and may not be correct. But users should expect that. In theory it applies to libraries called from a vector context as well. But in practice the calling code will either not be sensitive to such reordering, or the library routines will have been written with the explicit expectation of being used in a vector context.

2. A narrow majority of SG1 previously favored the explicit model to largely avoid this issue. By requiring explicit barriers of some sort, the implicit compiler restrictions disappear. But the problem with this is pointed out by the above example: there is no natural place to just put a barrier. In fact it would have to order a textually later operation before an earlier one. One would have to break the loop body up into multiple statements. This was previously pointed out, and SG1 was mostly convinced that this is a serious practical issue, at least for those already familiar with the implicit model.

3. There was brief thinking about alternative non-barrier-like syntax to address the problems with (2). But there wasn't much enthusiasm for trying to invent something new at this stage.

The Vector Type & Operations Details

This paper describes a template class for portable SIMD Vector types. The class is portable because its size depends on the target system and only operations that are independent of the SIMD register size are part of the interface.

The `Vector<T>` type only has the vector element type `T` as a parameter. It has `constexpr` members `MemoryAlignment` and `Size`.

The following APIs are supported:

- `load` and `store` based on a pointer to the scalar element type.
  - With optional mask.
  - Some are based on a pointer to a different scalar type, leading to conversion.
  - Optional flags specify alignment, temporality, and prefetching.
- Unary `+` and `-`.
- Binary arithmetic, comparison, bitwise, and shift.
- Subscripting.
- Gather and scatter.

Acknowledgement

Thanks to Chandler Carruth, Joel Falcou, Michael Wong, and Robert Geva for their review of the pre-publication paper.

Thanks to the many vector programming paper authors and SG1 for working tirelessly on such a complex topic for years.